

Lexical & Semantic Retrieval on MIND and EB-NeRD

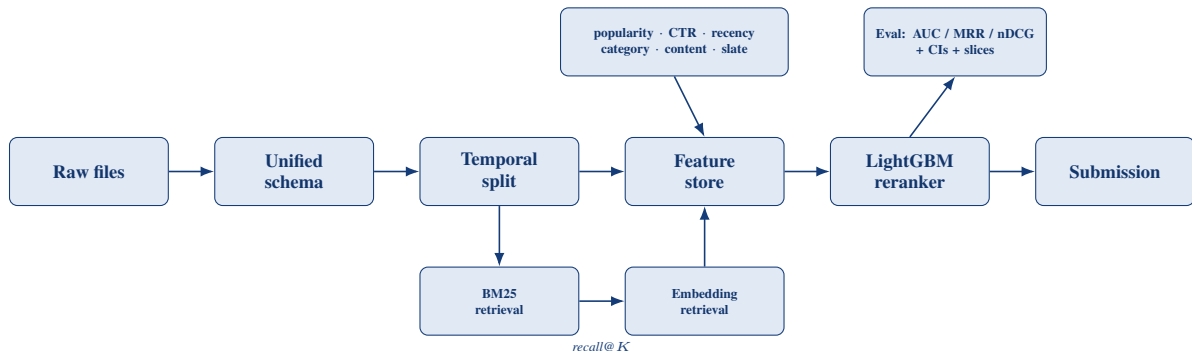
A measured, reproducible news-recommendation pipeline

Shivam Patel | Roll No. 2025202030 | CS4.406 Information Retrieval & Extraction — Assignment 1

github.com/shivam-git-acc/IRE_ASS1

1 What We Built

A reproducible pipeline that ranks the candidate articles of each impression by click likelihood, across two datasets—**MIND** (English, Microsoft) and **EB-NeRD** (Danish, Ekstra Bladet)—kept in separate namespaces (`mind:`, `eb:`) and *never merged* (different languages, separate leaderboards). One command rebuilds everything from raw files.



The pipeline parses both datasets into a unified schema, splits strictly by *time* (never random), builds *leakage-safe* point-in-time features, retrieves candidates (BM25 + embeddings, $\text{recall}@K$), and reranks the in-view slate with a LightGBM LambdaMART model. The evaluation harness reports AUC, MRR, nDCG@5/10, plus diversity, novelty and coverage with bootstrap 95% CIs and head/tail slices. A leakage assertion (Q9) confirms no history timestamp is $\geq$ its impression time (0 violations). **Leaderboards (Q5): MIND AUC 0.6435, EB-NeRD AUC 0.6976.**

1.1 Unified schema and reproducibility

The two datasets differ substantially at the raw level, so the schema mapping is where most correctness work lives. MIND ships tab-separated behaviours with an impressions string (`news_id-label` tokens) and a space-separated, *order-only* history (no timestamps). EB-NeRD ships Parquet with an explicit `article_ids_inview` candidate list, a separate `article_ids_clicked` label list, and a history table carrying `impression_time_fixed` timestamps—which we exploit for an *exact* leakage filter. Two dataset-specific gotchas we encoded: EB-NeRD’s scalar `article_id` column is *context*, not a candidate (a classic mistake), and its testset ships double-nested (`ebnerd_testset/ebnerd_testset`). MIND-small has no article body, so body features are disabled there. All of this is captured in one `build_pipeline.py` entry point plus per-dataset YAML configs, so a single command rebuilds each dataset from raw archives.

1.2 Retrieval vs. reranking: two distinct tasks

We treat these as separate problems throughout, which the results below justify. *Retrieval* scores the entire article pool (12k–125k items) and takes the top- K ; $\text{recall}@K$ measures whether the clicked article is caught. *Reranking* scores only the given in-view slate (~ 8 –250 candidates) and reorders it; AUC/MRR/nDCG measure the ordering. The Codabench leaderboard is a reranking task—the candidate slate is provided—so ANN indexing is irrelevant to submission and matters only for the retrieval experiments.

2 Key Design Choices and Alternatives

Temporal vs. random splitting. Interaction data must be split by time—train on the past, test on the future—or a later impression leaks into training and inflates scores. Temporal splitting was not just correct but *diagnostic*: it exposed concept drift a random split would hide.

Point-in-time (rolling) vs. frozen features. For an impression at T , every behavioural count uses only events strictly before T (`bisect_left` over sorted per-article event times), recomputed per impression. Precomputing popularity once over the training window both leaks and goes stale across the train $\rightarrow$ test gap.

Headline finding 1 — temporal drift

On EB-NeRD, **frozen popularity scores AUC 0.568; rolling point-in-time popularity scores 0.773** (+0.205). Frozen statistics leak *and* drift; the rolling recomputation is both leakage-safe and far stronger. This doubles as our Q9 with-vs-without serving-time-feature ablation.

Embedding model tracks language, not size. On English MIND, the *smaller* English-specialised all-MiniLM-L6-v2 (semantic AUC 0.671) beat the larger multilingual multilingual-e5-base (0.600)—multilingual capacity is diluted across ~ 100 languages. We use MiniLM for MIND and paraphrase-multilingual-MiniLM-L12-v2 for Danish EB-NeRD. Correcting E5’s asymmetric query:passage prefixes added +0.014; swapping E5→MiniLM lifted the MIND leaderboard 0.6292 $\rightarrow$ 0.6435.

Headline finding 2 — specialisation beats size

An English-specialised 6-layer MiniLM (+0.07 AUC) outperformed a larger multilingual E5 on English news. Model–language alignment mattered more than parameter count or embedding dimension.

Feature-based LightGBM vs. a neural recommender. We trained NRMS (neural, title-only) as an alternative; it reached dev AUC 0.662, *below* our feature LightGBM (0.695–0.697). Feeding NRMS as a LightGBM feature did not help: raw logits hijacked the tree splits (gain importance $28\times$ all other features $\rightarrow$ collapsed to NRMS-alone 0.665); per-slate rank-normalisation recovered only to 0.6755. NRMS’s content signal is redundant with our BM25/embedding features while lacking the behavioural signals that dominate. *Feature engineering beat the neural model and their fusion.*

Brute-force retrieval vs. ANN. At 12k–125k articles, exact brute-force cosine search is fast and returns exact neighbours (the assignment permits it); FAISS/ScaNN are the $10\times$ answer (§5), not needed at our scale.

3 Observations

3.1 Lexical vs. semantic — content is weak for news

Semantic retrieval beats lexical on MIND at every K (Table 1), yet both are *weak in absolute terms* (~ 0.02 – 0.04 recall@200) on *both* datasets, robust to aggregation (MIND MiniLM mean-pool 0.0285 $\approx$ best-match 0.0290). The clicked article is rarely the content-nearest-neighbour of a user’s history—news engagement is driven by freshness and popularity, not similarity to past reads.

Table 1: Retrieval recall@ K from the full pool (single-click $\Rightarrow$ hit@ K).

method	MIND (English)			signal	EB-NeRD (Danish, demo)		
	@50	@100	@200		@50	@100	@200
BM25	.0075	.0135	.0210	recency	.914	.944	.960
MiniLM (mean)	.0125	.0190	.0285	popularity	.157	.264	.436
MiniLM (best)	.0080	.0180	.0290	BM25/E5	.010	.019	$\sim .037$

On which slices does semantic win? Slicing the MIND comparison by user history length (cold/warm at the 20th/80th percentiles, computed from the data) shows the semantic advantage is *not* uniform—it is governed by history length (Table 2). For cold-start users (≤ 7 history clicks) MiniLM and BM25 are near-parity ($1.31\times$), but for warm users (≥ 52 clicks) MiniLM’s recall@200 is $2.78\times$ BM25’s. The mechanism is visible in the raw numbers: as history grows, BM25 *degrades* (0.0111, its worst) because the concatenated-title query becomes a diluted bag of hundreds of tokens, while MiniLM *improves* (0.0307, its best) because the mean-pooled embedding becomes a sharper user vector. Slicing by article popularity (head vs. tail) showed no comparable widening, so *history length, not article niche-ness, is the axis on which semantic’s advantage concentrates*. **On EB-NeRD the picture inverts entirely:** bootstrapped 95% CIs on the BM25–E5 recall difference span zero in *every* slice at every K —the two content methods are statistically *indistinguishable*, and the nominal “winner” flips with K . The MIND history-length mechanism does not replicate (Danish histories are long fixed arrays, and content barely predicts clicks to begin with). The reading is clean: where content carries signal (English MIND) *how* it is modelled matters (semantic $>$ lexical, more so with history); where clicks are popularity/recency-driven (Danish EB-NeRD) the choice of content model is a coin-flip.

Table 2: MIND recall@200 by user-history slice (MiniLM vs. BM25); ratio = MiniLM/BM25.

User slice	n	BM25	MiniLM	ratio
cold (≤ 7 clicks)	875	0.0183	0.0240	$1.31\times$
mid	2311	0.0212	0.0234	$1.10\times$
warm (≥ 52 clicks)	814	0.0111	0.0307	$2.78\times$

3.2 The signal-inversion finding

On EB-NeRD, **recency dominates retrieval (0.96 recall@200) but collapses to chance on reranking (AUC 0.503)**, while **popularity is the opposite** (0.436 retrieval $\rightarrow$ 0.693 reranking). The production system already filters the in-view slate to fresh articles (median clicked-article age 2.8h; 94.8% within 24h of publish), so recency cannot discriminate *among* candidates that are all fresh—though across the full pool it perfectly separates fresh from stale.

Headline finding 3 — a signal’s value is task-dependent

Recency is the best *retrieval* signal (0.96) yet useless for *reranking* (0.503); popularity inverts (0.44 $\rightarrow$ 0.69). The same feature can dominate one stage and be worthless at the next—retrieval and reranking must be designed and evaluated separately.

Logistic reranker coefficients confirm the reranking hierarchy: popularity +0.692 $\gg$ recency +0.378 > category +0.142 > content +0.130.

3.3 Cross-dataset contrast

Both datasets reach ~ 0.70 with a trained LightGBM, but for *different* reasons (Table 3). EB-NeRD is **popularity/recency-driven** (top feature `pop_24h`); MIND is **CTR- and content-driven** (top feature `ctr_total`, with `e5_best` third). Content is a useful *reranking* feature on English MIND (E5 0.588; MiniLM lifts the board) but near-random on Danish EB-NeRD (E5 ≈ 0.52)—even though content *retrieval* is equally weak on both.

Table 3: Cross-dataset comparison (best trained LightGBM).

Dimension	EB-NeRD (Danish)	MIND (English)
Best AUC (dev / board)	0.773 / 0.6976	0.695 / 0.6435
Dominant feature	raw popularity	click-through rate
Content in reranking	$\approx$ random (0.52)	useful (E5 0.588)
Semantic vs. lexical	tie	MiniLM/E5 > BM25
Head / tail	tail $\geq$ head (drift)	head > tail
Coverage	0.155	0.037

3.4 Reranking metrics and feature importance

Table 4 shows the Q4 harness on EB-NeRD (bootstrap 95% CIs, head/tail); the learned ranker cleanly beats every single signal, and the head/tail split reveals where accuracy concentrates. Table 5 lists LightGBM gain importances—popularity and CTR windows dominate, with content (`e5_best`) a secondary contributor on MIND only.

Table 4: Q4 reranking metrics, EB-NeRD (AUC; head/tail slices; nDCG@10).

Method	AUC (all)	head	tail	nDCG@10
LightGBM (rolling)	0.773	0.664	0.775	0.588
Popularity	0.693	0.825	0.568	0.545
Fusion (.4/.3/.3)	0.632	0.697	—	0.512
Content (BM25)	0.519	0.531	0.508	0.489
Recency	0.503	0.451	0.552	0.481

Table 5: LightGBM gain importance (top features per dataset).

Rank	EB-NeRD	MIND
1	<code>pop_24h</code>	<code>ctr_total</code>
2	<code>ctr_24h</code>	<code>ctr_24h</code>
3	<code>pop_1h</code>	<code>e5_best</code>
4	<code>slate_size</code>	<code>slate_size</code>
5	<code>recency</code>	<code>cat_affinity</code>

3.5 Beyond-accuracy: diversity, novelty, coverage

Accuracy alone hides how a ranker behaves. Our harness also reports intra-list diversity (1 – mean pairwise cosine among top-10), novelty ($-\log_2$ popularity of recommended items) and catalogue coverage. The contrast in *coverage* between datasets is telling: the EB-NeRD LightGBM recommends from a far wider slice of the catalogue (0.155) than the MIND model (0.037), because MIND’s CTR-dominated ranker concentrates on a small head of high-CTR articles, whereas EB-NeRD’s popularity+recency ranker spreads across many freshly-published items. This is a concrete accuracy–diversity trade-off: the more a model leans on a single sharp signal (CTR), the narrower its coverage.

3.6 Data scale and capacity

Training LightGBM on a 400k sample of MIND-large (AUC 0.697) $\approx$ full 2.2M $\approx$ MIND-small (0.695): **GBDT saturates**; data quantity is not the bottleneck. The dev $\rightarrow$ leaderboard drop (0.697 $\rightarrow$ 0.6435) is the *concept-drift tax*, not a data deficiency. A documented null result: session features changed EB-NeRD AUC by +0.001 (sessions are short—mean 1.9 impressions, 44% with >1), so intra-session context adds nothing at this session length—a useful negative result that saved us from over-engineering.

4 Where It Breaks at 10×

Retrieval. Brute-force $O(N)$ scoring is fine at 10^5 articles, not at 10^6 – 10^7 ; we would switch to an ANN index (FAISS IVF/HNSW, ScaNN), trading exact for sub-linear search. We already hit this: on the 13.5M-impression EB-NeRD testset, per-impression content scoring was infeasible and we fell back to article-level features.

Point-in-time features. Rolling popularity re-scans each article’s growing event list *per impression*. At $10\times$ traffic this dominates; production needs incremental windowed-aggregation stores (streaming counters with time-decay), not repeated list scans. Frozen precomputation is cheaper but—as shown—leaks and drifts, so the cost is unavoidable.

Memory / feature construction. Materialising feature matrices for millions of impressions crashed our workers; at $10\times$ this must be chunked/out-of-core or streamed, writing features and predictions incrementally. We adopted sampling (lossless given GBDT saturation) and incremental prediction writes.

Embedding refresh & drift. News catalogues churn hourly; at $10\times$ catalogue size the GPU cost of re-encoding new articles becomes binding, motivating incremental encoding of only newly-published articles plus an ANN index supporting online insertion. The frozen→rolling gap (0.568→0.773) shows popularity statistics must be refreshed continuously—a low-latency feature store updated in near real time, not a nightly batch.

5 Summary

Across both datasets, a disciplined feature-based pipeline—temporal splitting, leakage-safe point-in-time features, and a LightGBM reranker—outperformed a neural recommender and its hybrid, reaching AUC 0.6435 (MIND) and 0.6976 (EB-NeRD) on the leaderboards. Three findings generalise beyond this assignment. **(i) Concept drift is the dominant challenge in news recommendation:** frozen popularity statistics both leak and decay, and rolling point-in-time computation (+0.205 AUC on EB-NeRD) is essential rather than optional. **(ii) A signal’s value is task-dependent:** recency is the strongest retrieval signal yet useless for reranking, and popularity inverts—so retrieval and reranking must be designed and measured separately. **(iii) Model-data alignment beats raw capacity:** an English-specialised MiniLM beat a larger multilingual E5 on English news, and content that helps rerank English MIND is near-random on Danish EB-NeRD. The two datasets converge to similar accuracy (~ 0.70) through entirely different mechanisms—MIND via click-through and content, EB-NeRD via popularity and recency—a reminder that a recommender’s “why” matters as much as its score. The clearest lesson for scaling is that the honest engineering choices (temporal splits, rolling features, out-of-core construction) are exactly the ones that both prevent leakage *and* survive $10\times$ growth.

Leaderboard Submissions (Q5)

Both competitions were registered and submitted; Codabench score screenshots below.

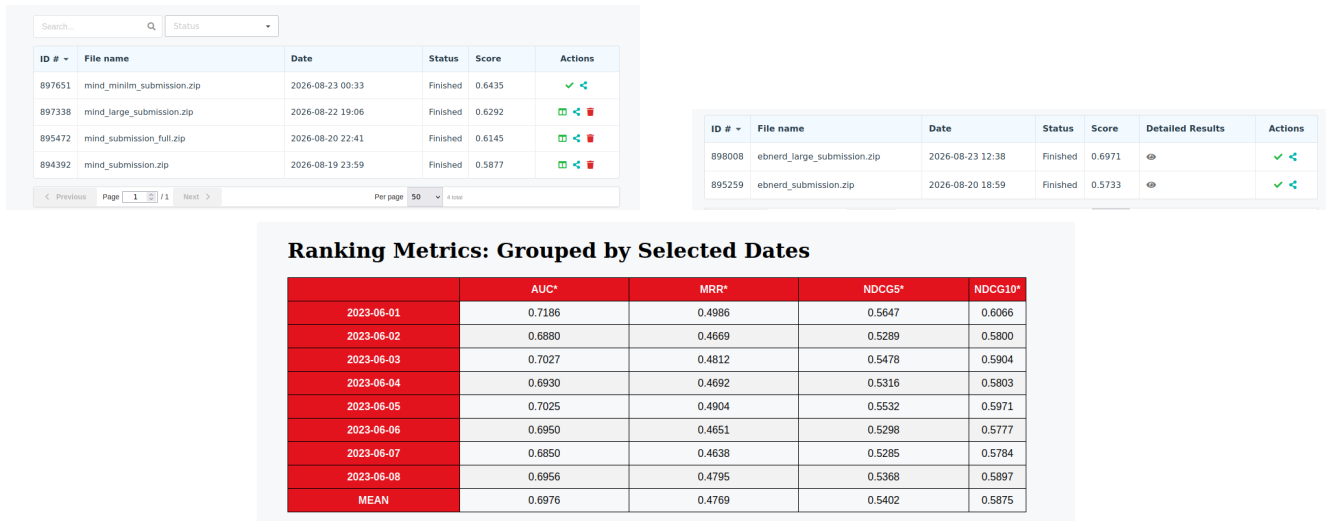


Figure 1: Codabench leaderboard scores. Top-left: MIND (comp. 13967) — AUC 0.6435, MRR 0.3141, nDCG@5 0.3411, nDCG@10 0.3976. Top-right: EB-NeRD (comp. 2469) — AUC 0.6976, MRR 0.4769, nDCG@5 0.5402, nDCG@10 0.5875. Bottom: EB-NeRD ranking detail.